Do Frozen Contextual Embeddings Improve Linear Classifiers for Sentiment Analysis?

Anonymous ACL submission

Abstract

We study whether frozen contextual embeddings from a pretrained transformer improve lightweight linear sentiment classifiers without requiring full fine-tuning. Using the IMDb movie review data set, we compare TF-IDF features against dense embeddings extracted from a frozen `bert-base-uncased` encoder, with Logistic Regression, Perceptron, and LinearSVC as downstream classifiers. We also compare two TF-IDF n-gram settings, two embedding pooling strategies, and a small qualitative error analysis. Our results show that TF-IDF with LinearSVC is the strongest model, outperforming all frozen BERT-based hybrids. This suggests that strong lexical baselines remain difficult to beat on IMDb sentiment classification, and that frozen contextual embeddings do not automatically yield better downstream performance when paired with simple linear classifiers.

1 Introduction

Sentiment analysis is a classic NLP task on which comparatively simple methods often remain highly competitive. Sparse lexical representations such as TF-IDF, combined with linear classifiers, are attractive because they are fast, interpretable, and effective when sentiment is expressed through salient words and short phrases.

At the same time, pretrained transformer encoders such as BERT provide contextualized representations that capture richer semantic information than surface lexical counts alone. In principle, these representations should help downstream classification. However, transformer-based pipelines are typically more expensive than standard lexical baselines, especially when models are fully fine-tuned.

This trade-off motivates our main question: can frozen contextual embeddings improve simple linear classifiers for sentiment analysis without the cost of end-to-end fine-tuning? To answer this, we compare strong TF-IDF baselines against a hybrid pipeline in which a frozen pretrained BERT encoder produces dense document embeddings that are then used by the same linear classifiers. We also test whether changing the TF-IDF n-gram range or the embedding pooling strategy changes the conclusion. This question is practically relevant because frozen feature extraction is often seen as a compromise between classical sparse pipelines and fully neural end-to-end models.

Our original hypothesis was that contextual embeddings from a frozen pretrained model would improve the accuracy of simple linear classifiers while remaining more efficient than full fine-tuning. Our experiments do not support this hypothesis. Instead, the strongest model is a classical lexical baseline: TF-IDF with LinearSVC. This result shows that strong lexical baselines should not be underestimated, and that contextual embeddings do not automatically yield better downstream performance when transferred in frozen, pooled form.

2 Related Work

Sentiment analysis has long been a central benchmark in NLP, and early successful systems often relied on sparse lexical features paired with linear classifiers. Bag-of-words, n-gram, and TF-IDF representations combined with models such as Logistic Regression or SVMs established strong baselines for document-level polarity classification and remain difficult to beat on many benchmarks, especially when sentiment is closely tied to explicit lexical cues (Pang et al., 2002).

Later work shifted toward distributed representations and neural models. Static word embeddings such as word2vec and GloVe improved generalization beyond sparse counts (Mikolov et al., 2013; Pennington et al., 2014), while contextualized representation models such as ELMo and

BERT achieved strong gains across many NLP tasks by representing words differently depending on context (Devlin et al., 2019).

A key distinction in work using pretrained language models is whether the encoder is used as a frozen feature extractor or fully fine-tuned end to end. Full fine-tuning often yields stronger task-specific performance, but it also increases computational cost and implementation complexity. Frozen feature extraction is simpler and cheaper, but the resulting representations may not align perfectly with the downstream task. Prior work has shown that pooling and transfer strategy can strongly affect downstream usefulness of pretrained representations (Reimers and Gurevych, 2019). Our project focuses on this setting: rather than proposing a new architecture, we ask whether frozen contextual embeddings provide a practical improvement over strong lexical baselines when the downstream model is kept intentionally simple.

3 Method

3.1 Task and Data

We study binary sentiment classification on the IMDb movie review data set. The IMDb movie review data set is a standard benchmark for sentiment classification (Maas et al., 2011). Each example is a movie review paired with a binary polarity label indicating either positive or negative sentiment. The full data set contains 50,000 reviews and is balanced across the two classes.

Rather than using the original pre-defined split directly, we reorganize the full corpus into an 80/10/10 train/validation/test split. This yields 40,000 training examples, 5,000 validation examples, and 5,000 test examples. We use a stratified split so that the class proportions remain consistent across all three partitions. The validation set is used for model comparison and ablation analysis, while the test set is reserved for final reporting.

IMDb is suitable for this comparison because it is a focused, text-only benchmark of moderate size, and because sentiment in movie reviews is often expressed through both explicit evaluative language and longer discourse structure. This makes it a reasonable setting in which to compare sparse lexical features against frozen contextual representations.

3.2 Preprocessing

Our preprocessing differs slightly depending on the feature pipeline.

For TF-IDF experiments, we apply lightweight normalization intended to reduce superficial variation while preserving sentiment-bearing lexical content. Specifically, we remove HTML break tags, remove punctuation, and lowercase the text before vectorization. We do not apply heavier linguistic preprocessing such as stemming or lemmatization because we want the baseline to remain simple and reproducible.

For contextual embedding experiments, we keep the original review text and pass it directly to the BERT tokenizer. This allows the pretrained encoder to operate on tokenized text in a manner closer to its original pretraining setup. Reviews are tokenized with truncation and padding, and embedding extraction is performed in batches on GPU so that the full data set can be processed efficiently.

3.3 Baseline Models

Our lexical baselines use a TF-IDF vectorizer with a maximum vocabulary size of 50,000. We test two feature settings:

- `ngram_range=(1,2)`
- `ngram_range=(1,3)`

The unigram–bigram setting serves as our main lexical baseline, while the trigram setting provides a small ablation to test whether a richer sparse representation changes the overall conclusion. In both cases, TF-IDF produces a high-dimensional sparse vector for each review.

On top of these sparse features, we train three standard linear classifiers:

- Logistic Regression
- Perceptron
- LinearSVC

We chose these models because they are standard, lightweight classifiers commonly used in text categorization. Comparing several linear classifiers also lets us separate the effect of the feature representation from the effect of the downstream decision rule.

3.4 Contextual Embeddings and Hybrid Models

For the hybrid systems, we use `bert-base-uncased` as a frozen feature extractor and do not fine-tune the encoder. Given an

input review, we run a forward pass through BERT and obtain the final hidden representations. We then convert these token-level outputs into a single fixed-dimensional document vector.

We test two pooling strategies. In the first setting, we use the final-layer `[CLS]` representation as the document embedding. In the second setting, we compute mean pooling over token representations after masking padding positions. This yields two 768-dimensional embedding spaces, one for each pooling method.

Using these embeddings as input features, we again train Logistic Regression, Perceptron, and LinearSVC. This design ensures that the downstream model family is held constant across the lexical and contextual settings. As a result, differences in performance can be attributed primarily to the representation itself rather than to a change in classifier complexity.

3.5 Implementation Details

For TF-IDF, we cap the vocabulary at 50,000 features. For BERT, we use `bert-base-uncased` as a frozen encoder and extract 768-dimensional document representations with batched GPU inference. Embeddings are precomputed for the train, validation, and test sets and saved to disk, so that downstream classifier training can be repeated without rerunning the encoder each time.

All downstream classifiers are trained with standard implementations from `scikit-learn`. We use default or near-default settings unless otherwise stated, since one goal of the project is to compare simple and reproducible model configurations rather than to perform an extensive hyperparameter search. This keeps the comparison focused on feature type, pooling strategy, and classifier family.

3.6 Evaluation

We use accuracy as the main metric because the IMDb data set is balanced and the task is binary classification. Validation accuracy is used during experimentation for comparing feature settings and pooling strategies, and test accuracy is used for final comparison.

In addition to accuracy, we inspect class-wise precision, recall, and F1 using classification reports during development. Although these values are not all included in the final table, they helped confirm that the main trends were not driven by severe class imbalance effects.

We also report classifier training time in seconds as a simple measure of efficiency. The reported training times correspond to downstream classifier training only and do not include the one-time cost of extracting frozen BERT embeddings. This distinction matters because the hybrid pipeline has an additional preprocessing stage that is absent from the lexical baseline.

4 Results

4.1 Main Results

Table 1 summarizes the main results. The best model overall is TF-IDF with LinearSVC using unigram and bigram features, which achieves 0.9140 validation accuracy and 0.9158 test accuracy. Logistic Regression is also strong, reaching 0.9100 validation accuracy and 0.9090 test accuracy with `ngram_range=(1,2)`.

The best frozen contextual hybrid is Logistic Regression on `[CLS]` embeddings, which reaches 0.8756 validation accuracy and 0.8808 test accuracy. Mean pooling performs similarly but slightly worse. LinearSVC on frozen embeddings is close to Logistic Regression but does not exceed it. Perceptron performs substantially worse on contextual embeddings.

These results show that the strongest sparse lexical baseline outperforms every frozen BERT-based hybrid model. In other words, contextual embeddings do not provide the expected gain in our lightweight transfer setting.

4.2 Ablation Trends

Our additional comparisons reveal three useful trends. First, increasing the TF-IDF feature space from `ngram_range=(1,2)` to `(1,3)` does not consistently improve performance. For LinearSVC and Logistic Regression, the unigram–bigram setting is slightly better on the test set.

Second, the difference between `[CLS]` pooling and mean pooling is small. For both Logistic Regression and LinearSVC, `[CLS]` pooling performs slightly better than mean pooling on validation and test data. This suggests that pooling strategy is not the main bottleneck in our hybrid setup.

Third, classifier choice matters differently across feature types. In the lexical TF-IDF setting, all three linear models perform reasonably well, with LinearSVC performing best. In the BERT embedding setting, Logistic Regression and Lin-

Model	Feature Type	Pooling / Setting	Validation Accuracy	Test Accuracy	Training Time (s)
LinearSVC	TF-IDF	ngram=(1,2)	0.9140	0.9158	0.7647
LinearSVC	TF-IDF	ngram=(1,3)	0.9104	0.9132	0.8623
Logistic Regression	TF-IDF	ngram=(1,2)	0.9100	0.9090	3.5602
Logistic Regression	TF-IDF	ngram=(1,3)	0.9114	0.9074	3.1408
Perceptron	TF-IDF	ngram=(1,3)	0.8960	0.8954	0.2516
Perceptron	TF-IDF	ngram=(1,2)	0.8904	0.8896	0.3150
Logistic Regression	BERT embeddings	CLS	0.8756	0.8808	7.0196
LinearSVC	BERT embeddings	CLS	0.8770	0.8802	12.5610
Logistic Regression	BERT embeddings	Mean	0.8744	0.8790	5.0136
LinearSVC	BERT embeddings	Mean	0.8728	0.8768	11.9558
Perceptron	BERT embeddings	Mean	0.7076	0.7116	0.5227
Perceptron	BERT embeddings	CLS	0.6708	0.6730	0.5210

Table 1: Main experimental results on the IMDb data set.

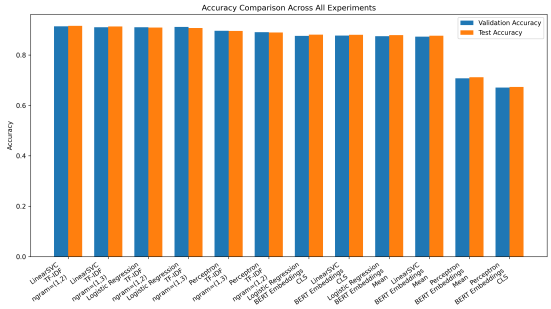

Figure 1: Validation and test accuracy across selected lexical and hybrid models.

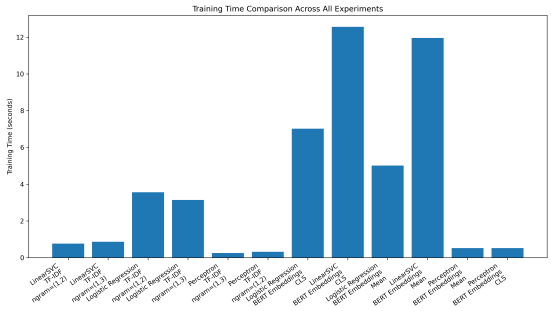

Figure 2: Classifier training time across selected lexical and hybrid models.

earSVC remain competitive, but Perceptron degrades sharply.

4.3 Figures

Figure 1 compares validation and test accuracy across the main models.

Figure 2 compares training time across the main systems. Although classifier training remains fast, these bars do not include the additional one-time cost of frozen embedding extraction.

Example review excerpts from disagreement cases
"... very convincing tale and interesting ... surprisingly good acting from all. Disadvantage – poor graphics. Does it matter? Nope." *Gold label: positive. TF-IDF + LR is correct; BERT CLS + LR is wrong.*
"... loved it ... ended with a typical cliffhanger ... leaving its fans feeling cheated. A waste of great writing and acting." *Gold label: negative. BERT CLS + LR is correct; TF-IDF + LR is wrong.*

Table 2: Example excerpts from test-set disagreement cases between TF-IDF + Logistic Regression and BERT CLS + Logistic Regression.

4.4 Qualitative Error Analysis

To better understand where the lexical and contextual pipelines differ, we performed a small qualitative analysis comparing TF-IDF with Logistic Regression against BERT [CLS] embeddings with Logistic Regression. We examined examples that TF-IDF classified correctly but the contextual hybrid misclassified, as well as examples in which the reverse occurred.

When TF-IDF was correct, the reviews often contained concentrated local lexical cues. Highly polarized words or short evaluative phrases seem easy for sparse features to preserve directly. When the contextual model was correct, broader discourse structure appeared to matter more, especially in reviews with mixed signals or delayed sentiment cues. This suggests that contextual representations do encode useful information, but not enough in our frozen, pooled setup to overcome the stronger overall advantage of strong lexical baselines.

5 Discussion and Conclusion

Our initial hypothesis was that incorporating frozen contextual embeddings from a pretrained transformer into a simple linear classifier would improve sentiment classification performance relative to standard TF-IDF baselines. The results do not support this hypothesis. Instead, the best-performing system is a classical baseline: TF-IDF with LinearSVC using unigram and bigram features.

This result is plausible for several reasons. IMDb sentiment classification relies heavily on lexical cues, and sparse TF-IDF features capture those patterns directly. Frozen BERT representations are generic rather than task-specific, so the extracted document vectors may not align optimally with the downstream decision boundary. In addition, pooling a document into a single fixed vector may compress away information that would otherwise help a linear classifier separate the two classes.

At the same time, the hybrid results are still informative. Logistic Regression and LinearSVC on frozen embeddings achieve test accuracy around 0.88, indicating that pretrained contextual representations do capture substantial sentiment-relevant information. The problem is not that the representations are useless, but that they do not outperform the strongest lexical alternative under our lightweight transfer setup.

Our additional experiments reinforce this conclusion. Changing the TF-IDF n-gram setting from $(1, 2)$ to $(1, 3)$ does not materially change the ranking of the models, and changing the embedding pooling strategy from `[CLS]` to mean pooling produces only small differences. This suggests that the main limitation is not the exact pooling strategy or n-gram range, but the fact that frozen contextual transfer does not beat a strong lexical baseline on this task. This is a useful negative result, because it highlights that stronger pretrained representations do not automatically translate into better performance under lightweight transfer settings, and that robust lexical baselines remain essential points of comparison.

Our study has several limitations. We tested only one pretrained encoder, restricted ourselves to frozen feature extraction, and focused on a single data set. Natural extensions would be to fine-tune BERT directly on IMDb, compare against another sentiment benchmark, or test stronger document embedding models.

In conclusion, our experiments suggest that strong lexical baselines remain highly competitive on IMDb sentiment classification and can outperform frozen contextual transfer when the downstream model is kept simple.

Statement of Contributions

All three team members contributed equally to project design, implementation, experimentation, analysis, and writing. The report and experiments were developed collaboratively, and all authors participated in revising the final submission.

Attribution of Data and Toolkits

We used the IMDb movie review data set (Maas et al., 2011). All downstream classifiers were implemented with `scikit-learn`, and contextual embeddings were extracted using the Hugging Face `transformers` and `datasets` libraries.

References

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, pages 4171–4186.

Andrew L. Maas, Raymond E. Daly, Peter T. Pham, Dan Huang, Andrew Y. Ng, and Christopher Potts. 2011. Learning word vectors for sentiment analysis. In *Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies*, pages 142–150.

Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg S. Corrado, and Jeffrey Dean. 2013. Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*.

Bo Pang, Lillian Lee, and Shivakumar Vaithyanathan. 2002. Thumbs up? sentiment classification using machine learning techniques. In *Proceedings of the 2002 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 79–86.

Jeffrey Pennington, Richard Socher, and Christopher D. Manning. 2014. Glove: Global vectors for word representation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1532–1543.

Nils Reimers and Iryna Gurevych. 2019. Sentence-bert: Sentence embeddings using siamese bert-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 3982–3992.